
microgpt_dpo.py — An Annotated Guide

Direct Preference Optimization in Pure Python

@nealrichter + Amazon Kiro

June 29, 2026

Abstract

This document walks through `microgpt_dpo.py`, which implements Direct Preference Optimization (DPO) on top of a LoRA-finetuned `microgpt` model. DPO aligns a language model to human preferences without training a separate reward model, using a closed-form loss derived from the RLHF objective. The implementation demonstrates: dual model inference (policy + frozen reference), preference pair training, and the complete DPO loss computation—all in pure Python with zero dependencies.

This is Stage 3 of the teaching pipeline: **pretrain** (`microgpt.py`) → **LoRA SFT** (`microgpt_sft.py`) → **DPO** (this file). An optional visualization module (`microgpt_viz.py`, invoked via `--viz`) renders ASCII loss sparklines and attention heat maps. An example training run is included as an appendix.

Source: https://github.com/nealrichter/research/blob/main/microgpt/microgpt_dpo.py

Contents

1	Overview	1
2	The DPO Loss	2
3	Dual Model Architecture	2
3.1	Policy (differentiable, with LoRA)	2
3.2	Reference (plain floats, no grad)	2
4	Log-Probability Computation	2
5	DPO Training Loop	3
6	Merge and Export	3
7	Why DPO Over RLHF?	4
8	Usage	4
	Appendix: Example Training Run	5

1 Overview

1. **Load:** SFT model as both frozen reference π_{ref} and policy init π_{θ} .
2. **Adapt:** LoRA on Q and V (policy only).
3. **Train:** DPO loss on (prompt, chosen, rejected) triples.

4. **Merge:** Fold adapters $\rightarrow$ `model_dpo.json`.

Parameter	Value
β (KL penalty)	0.3
Reference model	Frozen SFT checkpoint
LoRA rank	4
Learning rate	0.005
Epochs	30
Preference pairs	140

Table 1: DPO configuration

2 The DPO Loss

DPO [1] collapses the 3-stage RLHF pipeline into one loss:

$$\mathcal{L}_{\text{DPO}} = -\mathbb{E} \left[\log \sigma \left(\beta \left(\log \frac{\pi_{\theta}(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \log \frac{\pi_{\theta}(y_l|x)}{\pi_{\text{ref}}(y_l|x)} \right) \right) \right]$$

Derived from the RLHF objective: $\max_{\pi} \mathbb{E}[r(x, y)] - \beta \text{KL}[\pi \| \pi_{\text{ref}}]$.

The implicit reward is $r(x, y) = \beta \log \frac{\pi_{\theta}(y|x)}{\pi_{\text{ref}}(y|x)}$ — the model *is* the reward model.

3 Dual Model Architecture

3.1 Policy (differentiable, with LoRA)

```
105 state_dict = {k: [[Value(w) for w in row] for row in mat]
106                 for k, mat in model['weights'].items() }
```

```
154 def gpt_policy(token_id, pos_id, keys, values):
155     x = [t + p for t, p in zip(state_dict['wte'][token_id],
156                               state_dict['wpe'][pos_id])]
157     x = rmsnorm(x)
158     for li in range(n_layer):
159         ...
160         q = linear_lora(x, state_dict[f'layer{li}.attn_wq'],
161                         lora_dict[f'layer{li}.attn_wq.down'],
162                         lora_dict[f'layer{li}.attn_wq.up'])
163         k = linear(x, state_dict[f'layer{li}.attn_wk'])
164         v = linear_lora(x, state_dict[f'layer{li}.attn_wv'],
165                         lora_dict[f'layer{li}.attn_wv.down'],
166                         lora_dict[f'layer{li}.attn_wv.up'])
```

3.2 Reference (plain floats, no grad)

The reference uses raw floats for $\sim 3\text{--}5\times$ speedup (no autograd overhead):

```
116 ref_weights = {k: [[w for w in row] for row in mat]
117                   for k, mat in model['weights'].items() }
```

```

186 def linear_ref(x, w):
187     return [sum(wo[j] * x[j] for j in range(len(x))) for wo in w]
188
189 def gpt_ref(token_id, pos_id, keys, values):
190     # identical architecture, plain float arithmetic (no Value nodes)
191     ...

```

4 Log-Probability Computation

$$\log \pi(y|x) = \sum_{t=1}^{|y|} \log P(y_t | x, y_{<t})$$

```

231 def log_prob_policy(prompt_tok, resp_tok):
232     """Returns Value (differentiable)"""
233     tokens = prompt_tok + resp_tok
234     n = min(block_size, len(tokens) - 1)
235     keys, values = [[] for _ in range(n_layer)], [[] for _ in range(n_layer)]
236     log_p = Value(0.0)
237     resp_start = len(prompt_tok)
238     for pos_id in range(n):
239         logits = gpt_policy(tokens[pos_id], pos_id, keys, values)
240         if pos_id >= resp_start - 1 and pos_id < len(tokens) - 1:
241             probs = softmax(logits)
242             log_p = log_p + probs[tokens[pos_id + 1]].log()
243     return log_p
244
245 def log_prob_ref(prompt_tok, resp_tok):
246     """Returns float (frozen, no grad)"""
247     ...
248     log_p += math.log(probs[tokens[pos_id + 1]] + 1e-10)
249     ...

```

5 DPO Training Loop

```

358     # Policy log probs (differentiable)
359     log_pi_chosen = log_prob_policy(prompt_tok, chosen_tok)
360     log_pi_rejected = log_prob_policy(prompt_tok, rejected_tok)
361
362     # Reference log probs (frozen, float)
363     log_ref_chosen = log_prob_ref(prompt_tok, chosen_tok)
364     log_ref_rejected = log_prob_ref(prompt_tok, rejected_tok)
365
366     # DPO loss
367     reward_chosen = log_pi_chosen + Value(-log_ref_chosen)
368     reward_rejected = log_pi_rejected + Value(-log_ref_rejected)
369     logit = (reward_chosen + (reward_rejected * -1)) * beta
370     loss = (sigmoid(logit).log()) * -1

```

The loss decreases when π_θ assigns relatively higher probability to chosen over rejected (vs. reference).

6 Merge and Export

$$W_{\text{merged}} = W_0 + \frac{\alpha}{r} \cdot W_{\text{up}} \cdot W_{\text{down}}$$

```
399     for i in range(n_layer):
400         for key in (f'layer{i}.attn_wq', f'layer{i}.attn_wv'):
401             w_base = state_dict[key]
402             w_down = lora_dict[f'{key}.down']
403             w_up = lora_dict[f'{key}.up']
404             for r in range(len(w_base)):
405                 for c in range(len(w_base[0])):
406                     w_base[r][c].data += lora_scale * sum(
407                         w_up[r][k].data * w_down[k][c].data
408                     for k in range(lora_rank))
```

7 Why DPO Over RLHF?

	RLHF (PPO)	DPO
Stages	3 (SFT → RM → PPO)	1
Reward model	Required	Implicit
Stability	Sensitive	Stable
Memory	4 models	2 models

Table 2: RLHF vs. DPO (Rafailov et al. [1])

8 Usage

```
1 python3 microgpt.py           # pretrain
2 python3 microgpt_sft.py       # LoRA SFT
3 python3 microgpt_dpo.py       # DPO -> model_dpo.json
4 python3 microgpt_dpo.py -i    # inference from model_dpo.json
5 python3 microgpt_dpo.py -n 500 # cap at 500 steps
6 python3 microgpt_dpo.py --viz 100 # with visualization
7 python3 microgpt_dpo.py -h    # help
```

References

- [1] Rafailov, R., et al. *Direct Preference Optimization: Your Language Model is Secretly a Reward Model*. NeurIPS, 2023. <https://arxiv.org/abs/2305.18290>
- [2] Tie, G., et al. *A Survey on Post-training of Large Language Models*. arXiv:2503.06072, 2025. <https://arxiv.org/abs/2503.06072>
- [3] Hu, J.E., et al. *LoRA: Low-Rank Adaptation of Large Language Models*. arXiv:2106.09685, 2021. <https://arxiv.org/abs/2106.09685>
- [4] Ouyang, L., et al. *Training Language Models to Follow Instructions with Human Feedback*. NeurIPS, 2022. <https://arxiv.org/abs/2203.02155>
- [5] Kingma, D.P. and Ba, J. *Adam: A Method for Stochastic Optimization*. ICLR, 2015. <https://arxiv.org/abs/1412.6980>

Appendix: Example Training Run

TBA